Studio Nx

Backend(FastAPI/Python) 37 — update_me()

2026-10-07

Backend(FastAPI/Python) 37 — `update_me()` (+ `MeUpdateRequest`)

항목	내용
파일	`studionx/backend/app/services/user_service.py` · `app/schemas/user.py`
작업	P1-08 회원 정보 — 닉네임 변경 · PRD 01 FR-106
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	36 `to_me_response()` · 38 `save_profile_image()`

1. 하는 일

내 닉네임을 바꾼다. 가입 때와 같은 규칙(2~20자, 다른 사람과 겹치지 않음, 탈퇴 3일 규칙)을 지킨다. 이메일은 바꿀 수 없다(FR-106).

지금 닉네임 → 보낸 값	결과
`my_name` → (칸 없음) · `null` · `"my_name"`	아무것도 안 함
`old_name` → `" New_Name "`	`New_Name` 으로 저장 (앞뒤 공백 제거)
`Demo_Case` → `"demo_case"` (대소문자만)	저장 — 내 닉네임이므로
`my_name` → `"taken_name"` (남이 `Taken_Name` 사용 중)	409 `NICKNAME_TAKEN` "This nickname is already in use."
`my_name` → `"gone_nick"` (어제 탈퇴한 회원의 닉네임)	409 `REJOIN_TOO_SOON` "This nickname can be used again after Oct 9, 2026 (UTC)."
`my_name` → `"free_again"` (4일 전 탈퇴한 회원의 닉네임)	저장. 탈퇴 회원 쪽은 `deleted_user_0000000085` 로 비켜 둠 (P1-04 `release_withdrawn`)
`" a "` · 21자	422 — 스키마에서 막힘 (아래)

2. 언제 쓰나

쓰는 곳	
`PATCH /users/me` (41)	본문 `{"nickname": "…"}` → 이 함수 → 36 으로 응답
화면	userInfo.html 의 닉네임 칸 · Check 버튼 · Save changes (P1-10)

템플릿에서 바꾼 것 2가지

	템플릿	바꾼 것	이유
스키마 `MeUpdateRequest.nickname`	`str` + 길이 2~20	가입과 같은 **`Nickname`** 타입	템플릿대로면 `" a "` 가 5자로 통과하고, 저장할 때 공백을 지우면 1자 닉네임이 생긴다. `Nickname` 은 공백을 먼저 지우고 길이를 잰다
"지금 닉네임과 다르면 확인"	글자가 다르면 확인	**DB 가 같다고 보면** 확인을 건너뜀	DB 비교 규칙(utf8mb4_0900_ai_ci)은 대소문자를 구분하지 않는다. `Demo_Case` → `demo_case` 를 확인하면 **내 닉네임**이 "사용 중" 으로 나와 바꿀 수 없다

3. 코드

import (파일 맨 위)

```
from pathlib import Path, PurePosixPath

from fastapi import UploadFile
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import User
from app.schemas.user import MeResponse, MeUpdateRequest
from app.services.auth_service import check_nickname
```

`select` · `IntegrityError` · `AppError` · `ErrorCode` · `check_nickname` 을 이번에 추가했다.

스키마 (`schemas/user.py`)

```
from app.schemas.auth import Nickname

class MeUpdateRequest(BaseModel):
    nickname: Nickname | None = None
```

함수

```
def update_me(db: Session, user: User, data: MeUpdateRequest) -> User:
    nickname = data.model_dump(exclude_unset=True).get("nickname")
    if nickname is None or nickname == user.nickname:
        return user

    mine = db.scalar(
        select(User.user_id).where(User.user_id == user.user_id, User.nickname == nickname)
    )
    if mine is None:
        result = check_nickname(db, nickname, release=True)
        if result.reason == "REJOIN_TOO_SOON":
            when = result.available_at
            raise AppError(
                ErrorCode.REJOIN_TOO_SOON,
                409,
                f"This nickname can be used again after {when:%b} {when.day}, {when.year} (UTC).",
                details={"nickname": "REJOIN_TOO_SOON", "available_at": when.isoformat()},
            )
        if not result.available:
            raise AppError(ErrorCode.NICKNAME_TAKEN, 409, "This nickname is already in use.")

    user.nickname = nickname
    try:
        db.commit()
    except IntegrityError:
        # 확인과 저장 사이에 다른 사람이 같은 닉네임으로 먼저 바꾼 경우
        db.rollback()
        raise AppError(ErrorCode.NICKNAME_TAKEN, 409, "This nickname is already in use.") from None
```

```
    db.refresh(user)
    return user
```

문법 풀이

import 문법	뜻
`from app.services.auth_service import check_nickname`	P1-04 (19) 의 닉네임 확인을 그대로 씀
`from sqlalchemy.exc import IntegrityError`	UNIQUE 등 DB 규칙 위반 오류

식	뜻
`Nickname | None = None`	닉네임 타입 또는 `None`, 안 보내면 `None`
`data.model_dump(exclude_unset=True)`	**보낸 칸만** 사전으로 — 보내지 않은 칸은 빠짐 (나중에 칸이 늘어도 안 보낸 칸은 안 바뀜)
`.get("nickname")`	없으면 `None`
`select(User.user_id).where(A, B)`	`WHERE A AND B` — 그 회원이면서 닉네임이 DB 규칙상 같은 행
`db.scalar(…)`	첫 행의 첫 값, 없으면 `None`
`f"{when:%b}"`	날짜를 `Oct` 처럼 달 이름으로
`raise … from None`	원래 DB 오류(IntegrityError)를 오류 기록에 붙이지 않음 — 예상한 상황이라서

4. 흐름

1. 보낸 닉네임을 꺼낸다. 없거나 지금과 똑같으면 그대로 돌려준다.
2. DB 가 "내 닉네임과 같다" 고 보는지 확인 (대소문자 · 악센트만 다른 경우).
3. 아니면 P1-04 의 닉네임 확인 → 탈퇴 3일 안이면 409 `REJOIN_TOO_SOON`, 사용 중이면 409 `NICKNAME_TAKEN`. (3일 지난 탈퇴 닉네임은 이 안에서 비켜 둠)
4. 저장. 그사이 남이 먼저 가져갔으면(UNIQUE 위반) 되돌리고 409.
5. DB 에서 다시 읽어 돌려준다.

5. 왜 이렇게

선택	이유
똑같으면 DB 를 건드리지 않음	화면이 닉네임을 안 바꾸고 Save 를 눌러도 쓸데없는 쓰기 · 확인이 없다
`null` 도 "안 바꿈"	닉네임을 비울 수는 없다 (DB `NOT NULL`). 오류보다 무시가 안전
탈퇴 3일 문구를 따로 만듦	P1-04 의 `unavailable_error` 는 "You can **sign up** with this nickname again after…" — 가입 문구다. 이미 검토가 끝난 함수라 고치지 않고 여기서 수정 화면에 맞는 문구를 만들었다
`release=True`	3일 지난 탈퇴 닉네임은 가입처럼 바로 쓸 수 있어야 한다 (FR-108). 비켜 두기는 같은 `commit` 으로 함께 저장되고, 409 로 끝나면 저장하지 않아 취소된다
`IntegrityError` → 409	P1-04 가입과 같은 이유 — 확인과 저장 사이의 짧은 틈
마지막에 `refresh`	DB 가 바꾼 값(`updated_at`)까지 받아 36 응답에 씀

6. 시험 (`tests/test_user_service.py` 에 14개 추가 · 모두 22개, 실제 MySQL)

시험	확인한 것
스키마	`" ab "` → `"ab"`, 20자 통과 / `" a "` · 21자 · 빈 글자 → 오류
바꿀 것 없음 (칸 없음 · `null` · 같음)	같은 객체를 돌려주고 DB 그대로
새 닉네임 `" New_Name "`	DB 에 `New_Name`
대소문자만 (`Demo_Case` → `demo_case`)	저장됨
남의 닉네임 (대소문자만 다름)	409 `NICKNAME_TAKEN`, 내 닉네임 그대로
어제 탈퇴한 회원의 닉네임	409 `REJOIN_TOO_SOON`, 문구 · `details`, 내 닉네임 그대로
4일 전 탈퇴한 회원의 닉네임	저장됨, 탈퇴 회원은 `deleted_user_{번호 10자리}`
동시에 바꾸기 (확인을 통과시켜 흉내)	`commit` 의 UNIQUE 위반 → 409, 내 닉네임 그대로

전체 시험 256개 통과, `ruff` 검사 통과. 시험이 끝난 뒤 DB 에는 데모 계정과 사용자 계정만 남음.